SDN

Introducción a los Sistemas Distribuidos (75.43)

Universidad de Buenos Aires, Facultad de Ingeniería

SDN

- ¿Qué es una SDN?
- ¿Dónde se implementan?
- ¿Para qué se utilizan?

Pero antes...

¿Qué es el **Data Plane**?

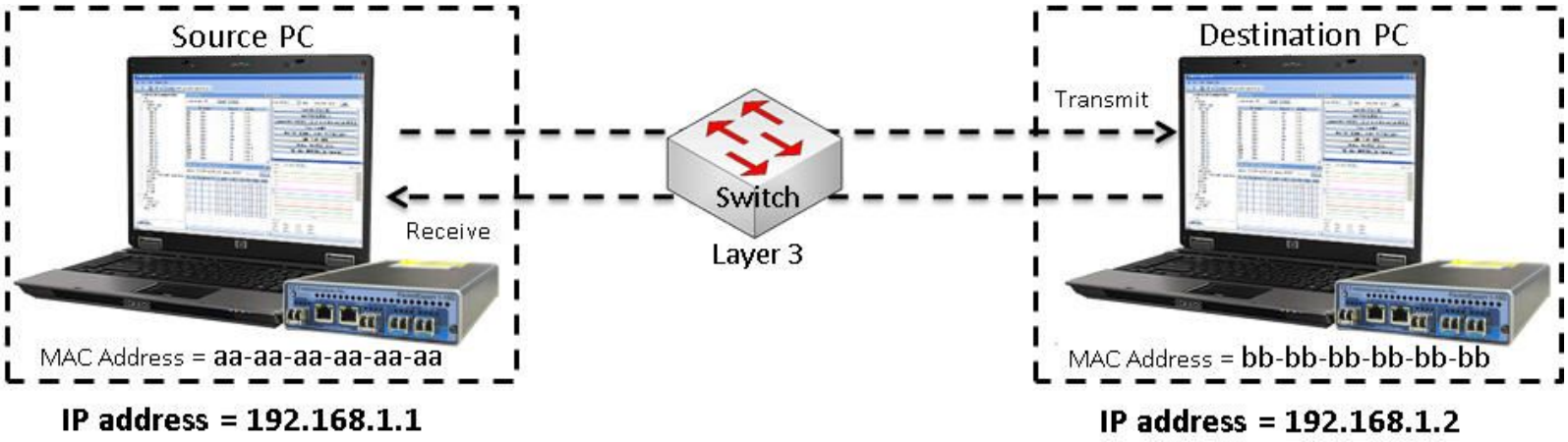
Es una parte del software que se encarga de ejecutar acciones para que el paquete llegue a destino

¿Qué es el **Control Plane**?

Es una parte del software que se encarga de determinar rutas para que paquete alcance el destino

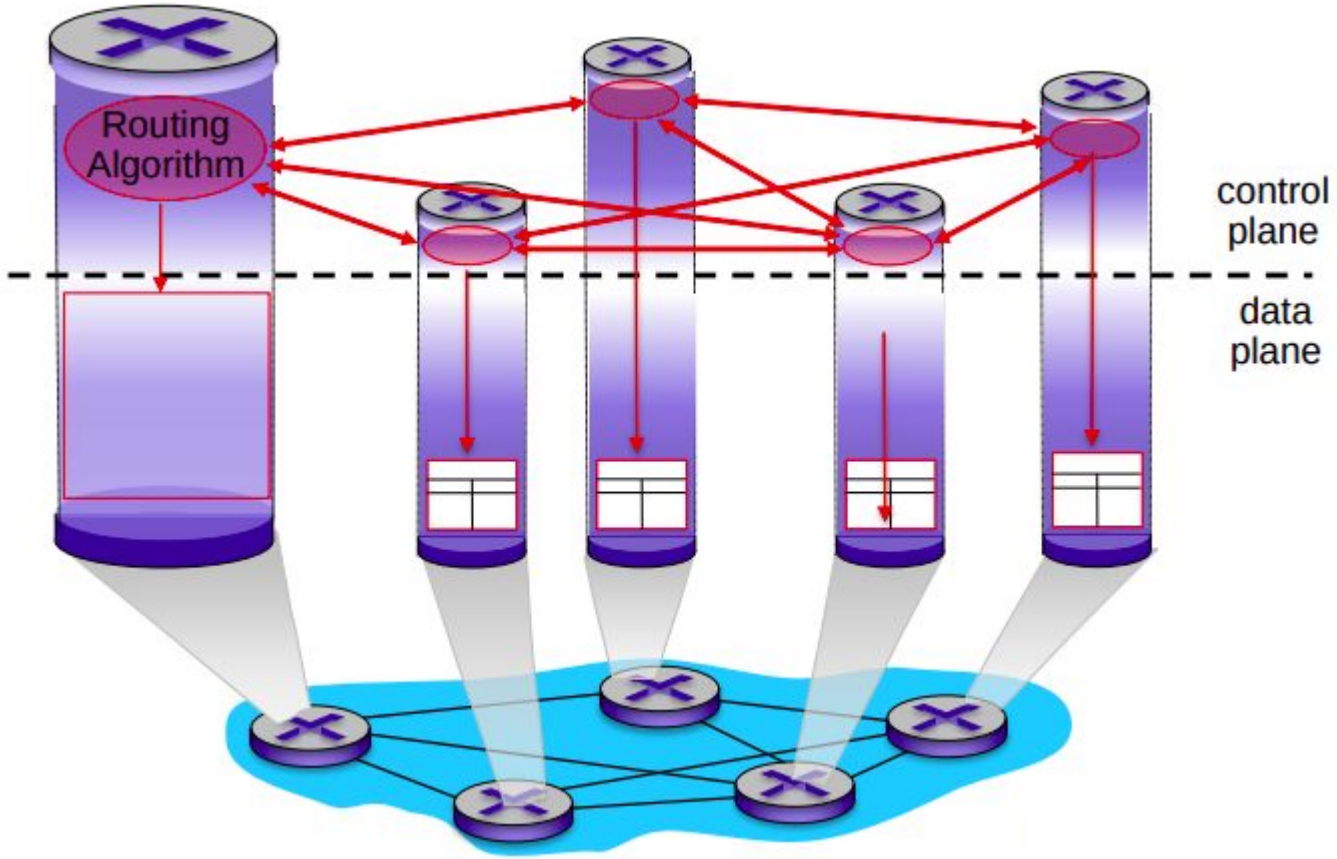
SDN

- ¿Cuál es el objetivo de la capa de red?
- ¿Y qué relación tiene con el Data Plane y Control Plane?



SDN

Los algoritmos de ruteo en cada uno de los routers interactúa con los demás en el control plane para computar las forwarding tables



SDN

- Esta separación de **Data Plane** y **Control Plane** tradicionalmente se encuentran en un mismo dispositivo (router).

- A mediados de 2008 en adelante, apareció la ruptura...

SDNs

SDN

- ¿Qué es una SDN?
  Es el concepto de separación de Control Plane y Data Plane

- ¿Para qué se utilizan?

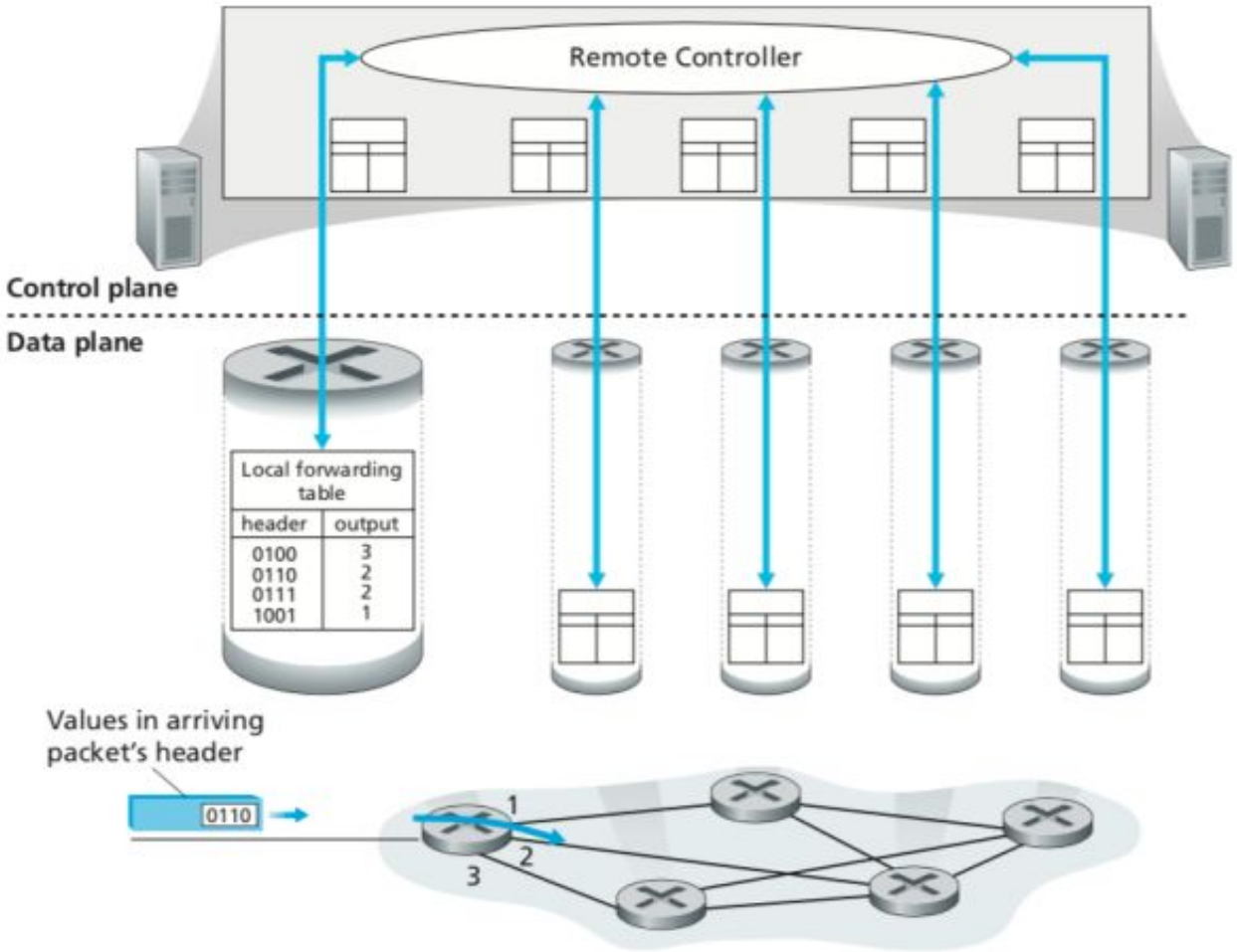
SDN

- ¿Qué es una SDN?
  Es el concepto de separación de Control Plane y Data Plane

- ¿Para qué se utilizan?
  Como se llaman Software Defined Network, las Forwarding Tables se definen por software

SDN

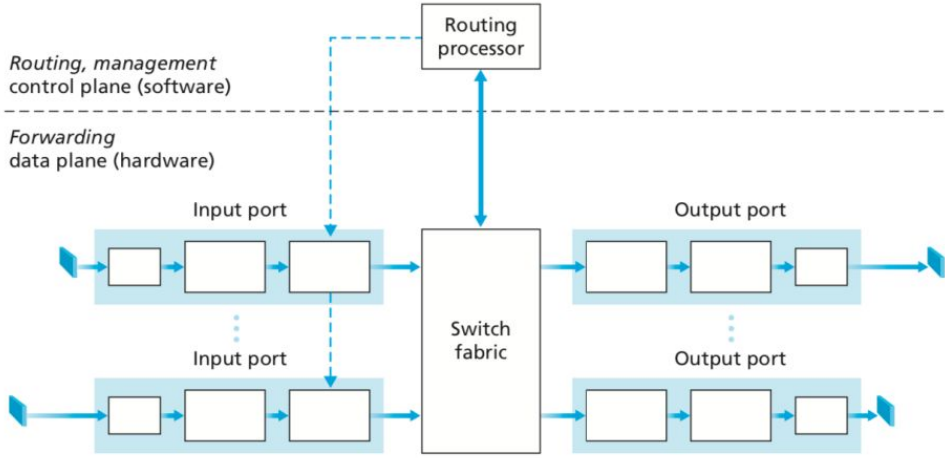
Un controlador remoto generalmente interactúa con los routers para poder computar las forwarding tables



SDN

Elementos dentro de un router:

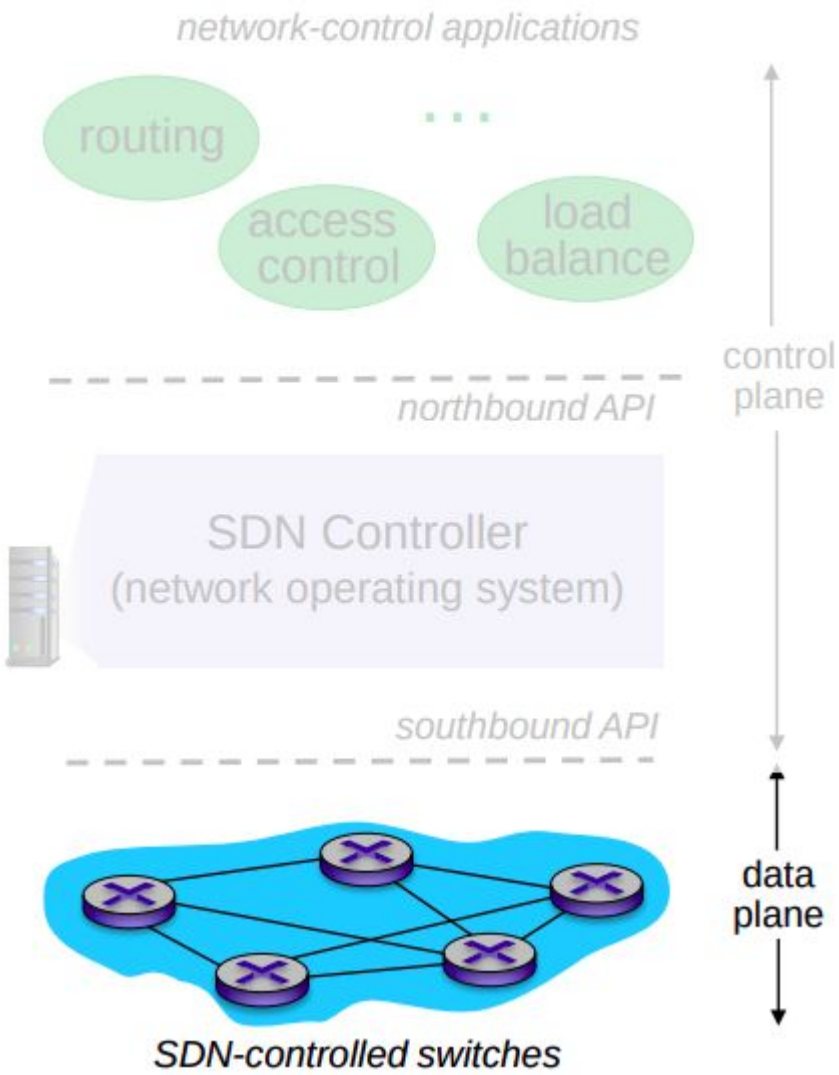
1. Input Ports
2. Output Ports
3. Switch Fabric
4. CPU



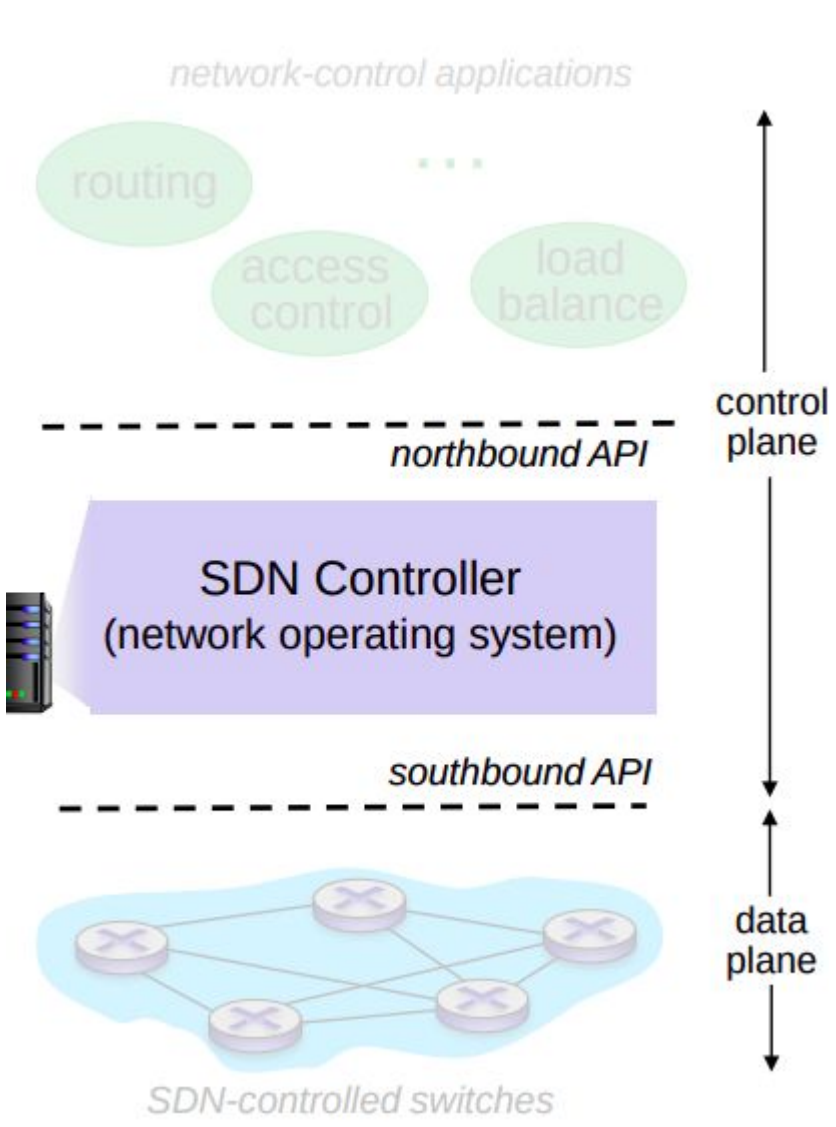
SDN

Switches Data Plane:

- Rápidos, simples, e implementan un forwarding generalizado del data plane
- Forwarding table instalada por el controller
- Utilizan un protocolo para comunicarse con el controller (ej Open Flow)



SDN



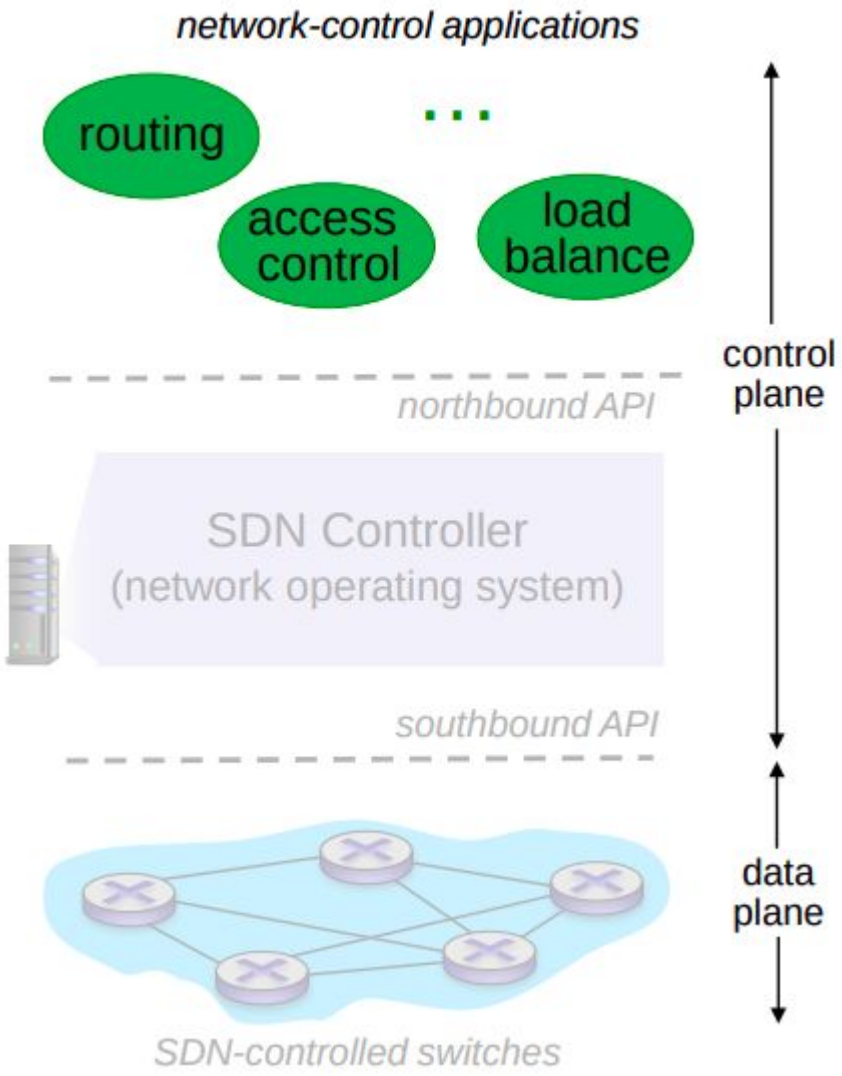
SDN controller:

- Mantiene información de la red
- Interactúa con las aplicaciones de control de red a través de la northbound API
- Interactúa con los switches a través de la southbound API
- Implementado como un sistema distribuido para mejorar performance, escalabilidad, tolerancia a fallas

SDN

Network control apps:

- Son el “cerebro” de control: implementan funciones de control
- API provided by SDN controller

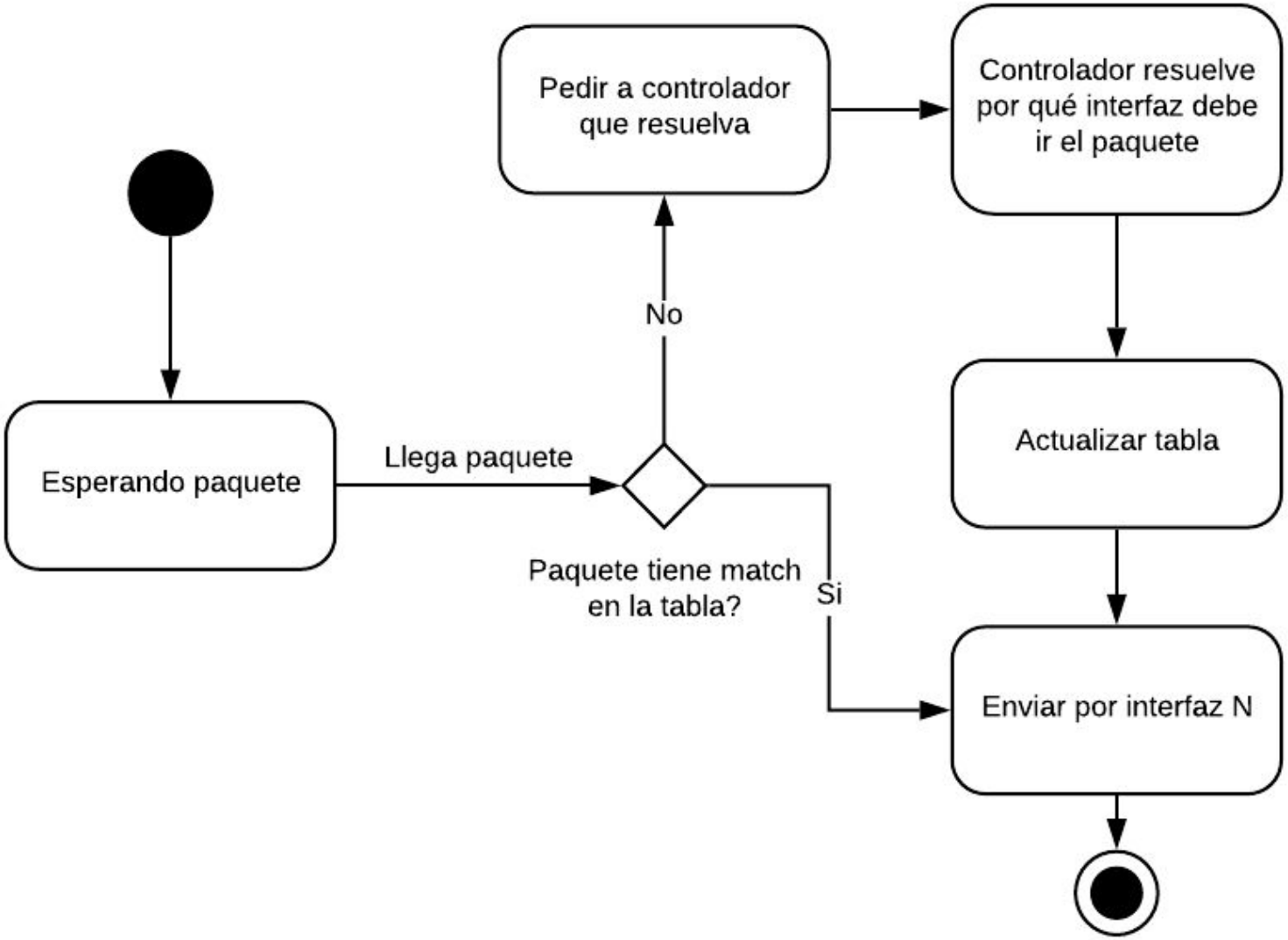


OpenFlow

OpenFlow

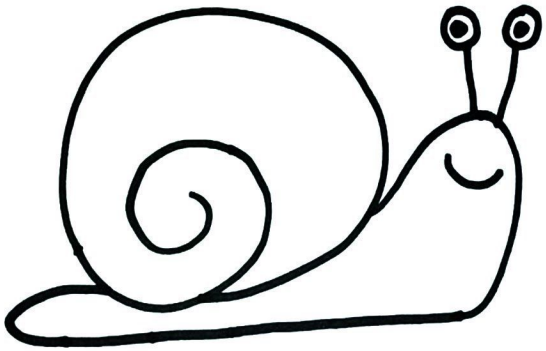
- Protocolo para interactuar con las tablas de CAM / routeo de los sw / routers
- Veamos un ejemplo de como funciona

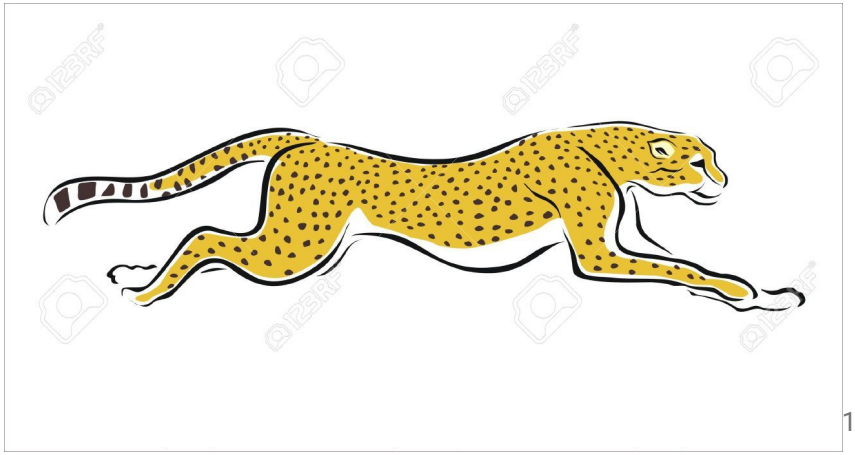
Estados de un SW OpenFlow



OpenFlow

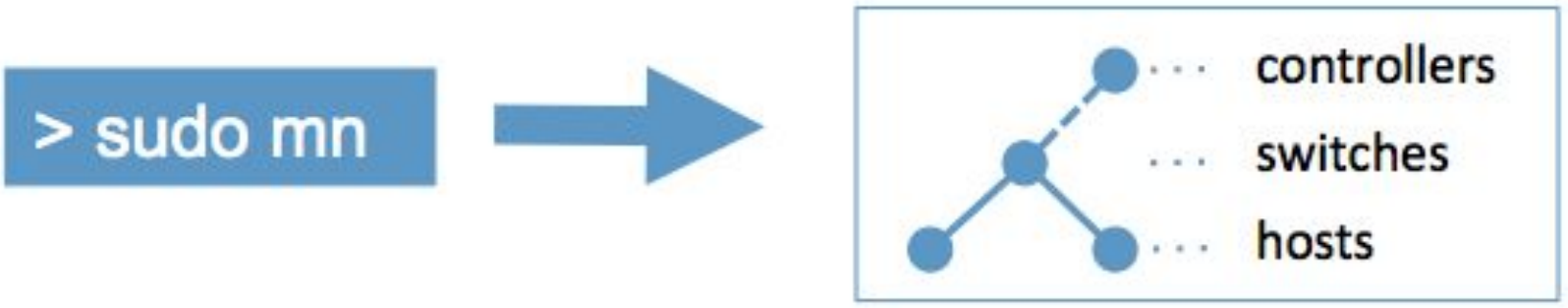
- Importante:
  - Actualizar las tablas luego de que el controlador resuelva
  - Las memorias de las tablas son muy rápidas
  - El controlador, no tanto





Mininet

Mininet



Crea una red virtual, ejecutando kernel, switch y código de aplicación, en una sola máquina.
Se utiliza para probar, desarrollar, y experimentar con OpenFlow y SDN.(1)

(1) http://mininet.org/

Mininet - Comandos

- **Lanzar mininet (CLI):**
  $ sudo mn
- **Listar nodos:**
  mininet> nodes
- **Listar enlaces:**
  mininet> links
- **Listar información de los nodos:**
  mininet> dump
- **Ejecutar comando en un device:**
  mininet> dev1 command
  mininet> h1 ifconfig -a
- **Probar conectividad entre hosts:**
  mininet> h1 ping h2
- **Abrir una xterm:**
  mininet> h1 xterm
- **Ping a todos:**
  mininet> pingall

Mininet - Ejemplo

Lanzar una instancia de mininet, sin parámetros. Analizar la topología y probar la conectividad entre dispositivos.

$ sudo mn

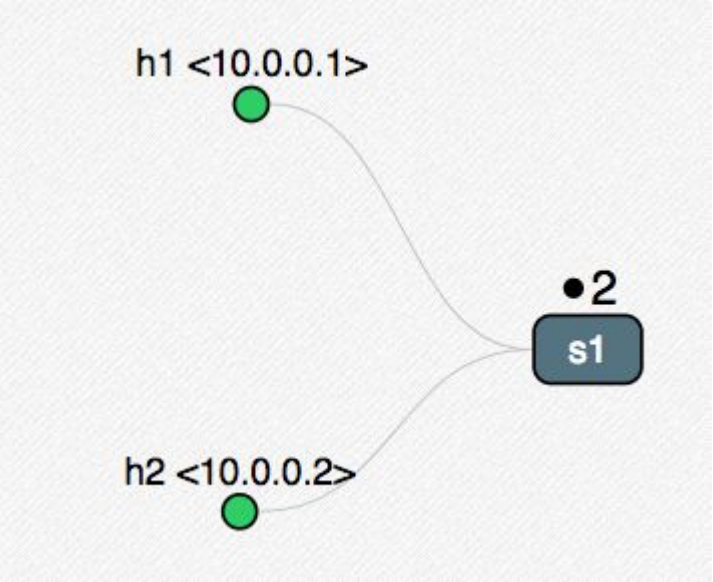
Por defecto, se crea una topología con un switch y dos hosts.

```
➜  ~ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Mininet - Ejemplo

```
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s1-eth2 (OK OK)

mininet> dump
<Host h1: h1-eth0:10.0.0.1
pid=1907>
<Host h2: h2-eth0:10.0.0.2
pid=1909>
<OVSSwitch s1:
lo:127.0.0.1,s1-eth1:None,s1-eth2
:None pid=1914>
<Controller c0: 127.0.0.1:6653
pid=1900>
```



Mininet - Ejemplo

Probar la conectividad entre los dispositivos

mininet> pingall
***** Ping: testing ping reachability**
h1 -> h2
h2 -> h1
***** Results: 0% dropped (2/2 received)**

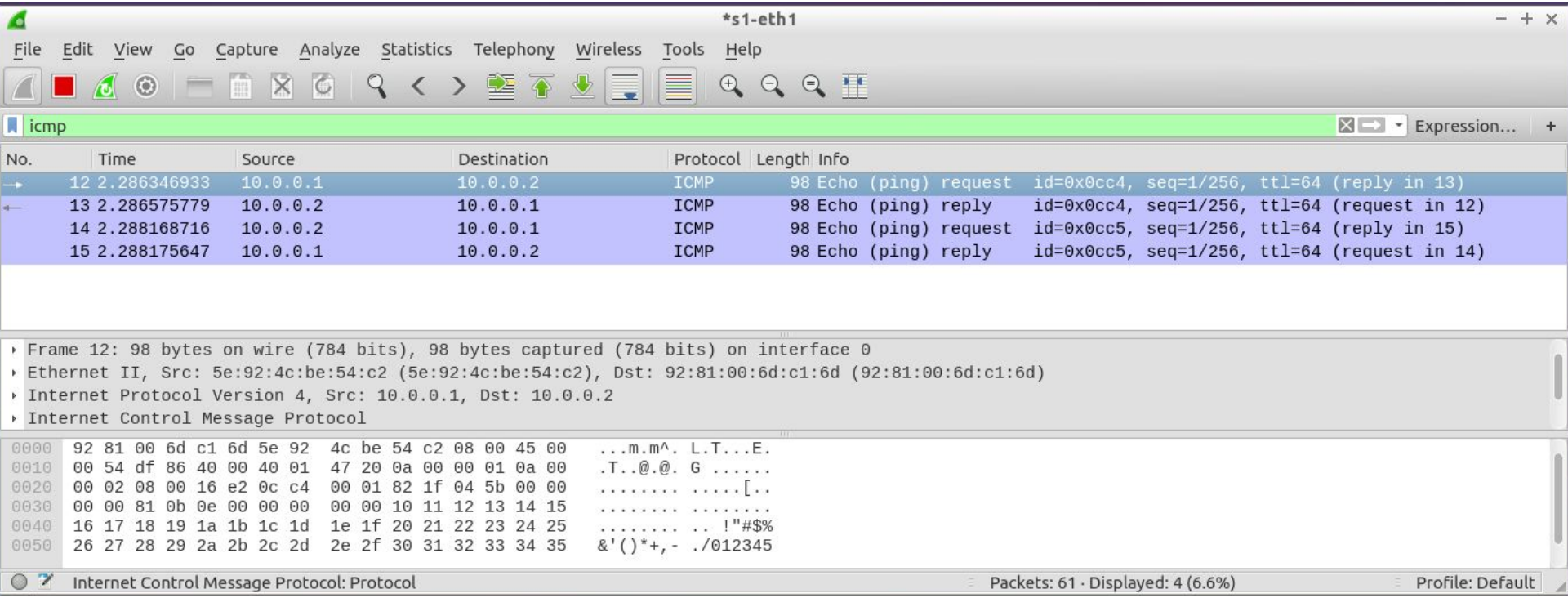
Otra forma

➜ **~ sudo mn --test pingall**

El comando pingall se utiliza para probar la conectividad entre todos los dispositivos, se ejecuta el comando ping en todos los dispositivos de la topología

> dev1 ping -c1 dev2

Mininet - Ejemplo



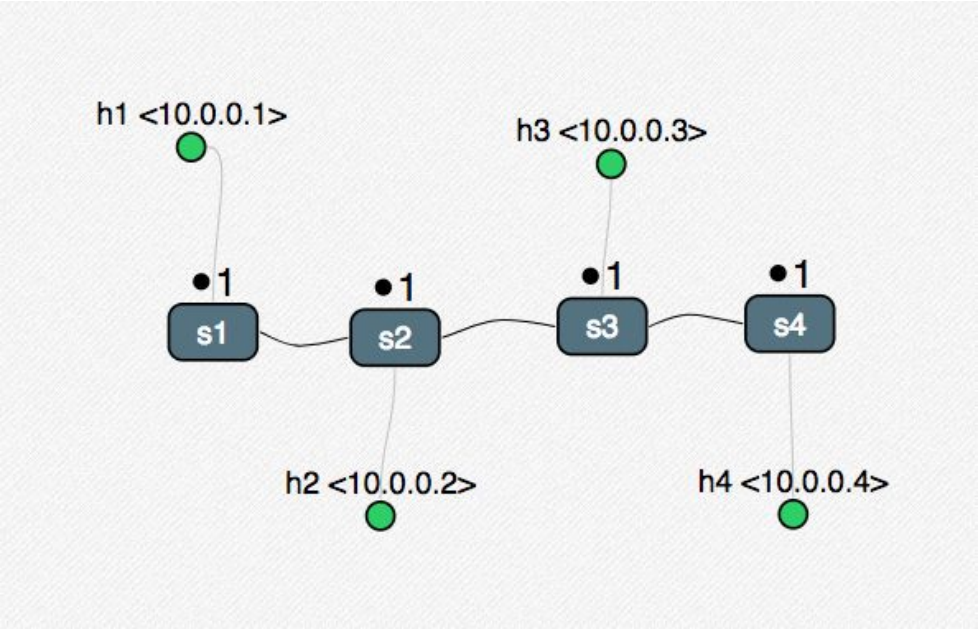
Mininet - Topologías más complejas

```
➜  ~ sudo mn --topo linear,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s2, s1) (s3, s2) (s4, s3)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet>
```

Topología "linear", recibe por parámetro la cantidad de switches, y crea una topología donde cada switch tiene un host conectado, y todos los switches están conectados en forma lineal.

Mininet - Topologías más complejas

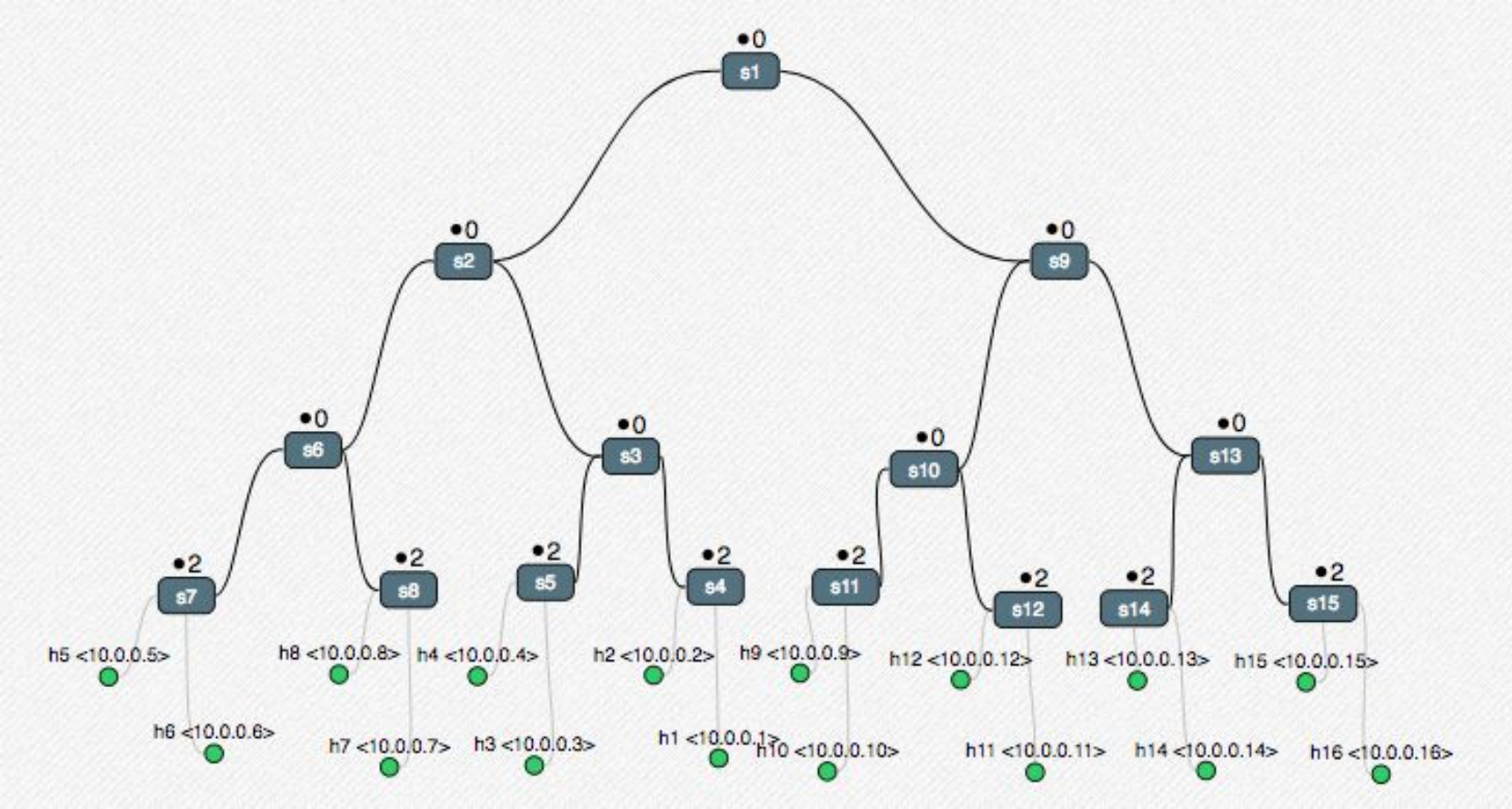
```
➜  ~ sudo mn --topo linear,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s2, s1) (s3, s2) (s4, s3)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet>
```



Mininet - Topologías más complejas

➜ ~ sudo mn --topo tree,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
*** Adding switches:
s1 s2 s3 s4 s5 s6 s7 s8 s9 s10 s11 s12 s13 s14 s15
*** Adding links:
(s1, s2) (s1, s9) (s2, s3) (s2, s6) (s3, s4) (s3, s5) (s4, h1) (s4, h2) (s5, h3) (s5, h4) (s6, s7) (s6, s8) (s7, h5) (s7, h6) (s8, h7) (s8, h8) (s9, s10) (s9, s13) (s10, s11) (s10, s12) (s11, h9) (s11, h10) (s12, h11) (s12, h12) (s13, s14) (s13, s15) (s14, h13) (s14, h14) (s15, h15) (s15, h16)
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
*** Starting controller
c0
*** Starting 15 switches
s1 s2 s3 s4 s5 s6 s7 s8 s9 s10 s11 s12 s13 s14 s15 ...
*** Starting CLI:
mininet>

Mininet - Topologías más complejas



Mininet - Topologías personalizadas

Mininet provee las siguientes topologías:

- linear
- minimal
- reversed
- single
- torus
- tree

Sin embargo, también se pueden crear topologías personalizadas usando la API Python que nos provee la herramienta.

Mininet - Topologías personalizadas

```
from mininet.topo import Topo

class MyTopo( Topo ):
   "Simple topology example."

   def __init__( self ):
      "Create custom topo."

      # Initialize topology
      Topo.__init__( self )

      # Add hosts and switches
      leftHost = self.addHost( 'h1' )
      rightHost = self.addHost( 'h2' )
      leftSwitch = self.addSwitch( 's3' )
      rightSwitch = self.addSwitch( 's4' )

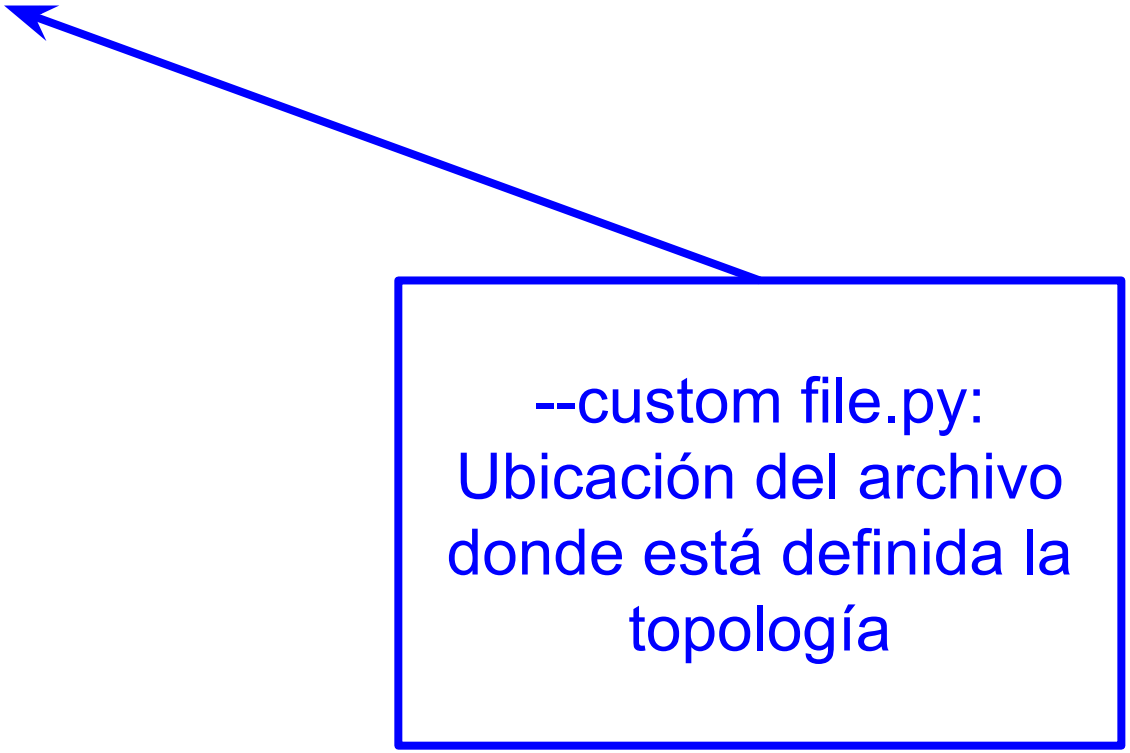
      # Add links
      self.addLink( leftHost, leftSwitch )
      self.addLink( leftSwitch, rightSwitch )
      self.addLink( rightSwitch, rightHost )


topos = { 'mytopo': ( lambda: MyTopo() ) }
```

custom/topo-2sw-2host.py

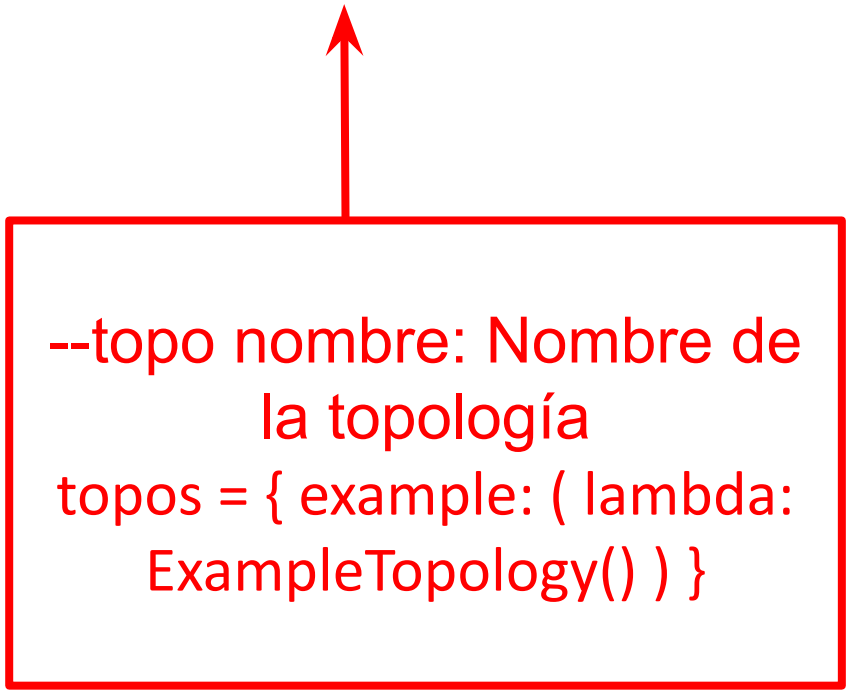
Mininet - Topologías custom

```
➜  ~ sudo mn --custom /tmp/topology/example.py --topo example --mac --switch ovsk
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
sw1 sw2
*** Adding links:
(h1, sw1) (sw1, sw2) (sw2, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
sw1 sw2 ...
```



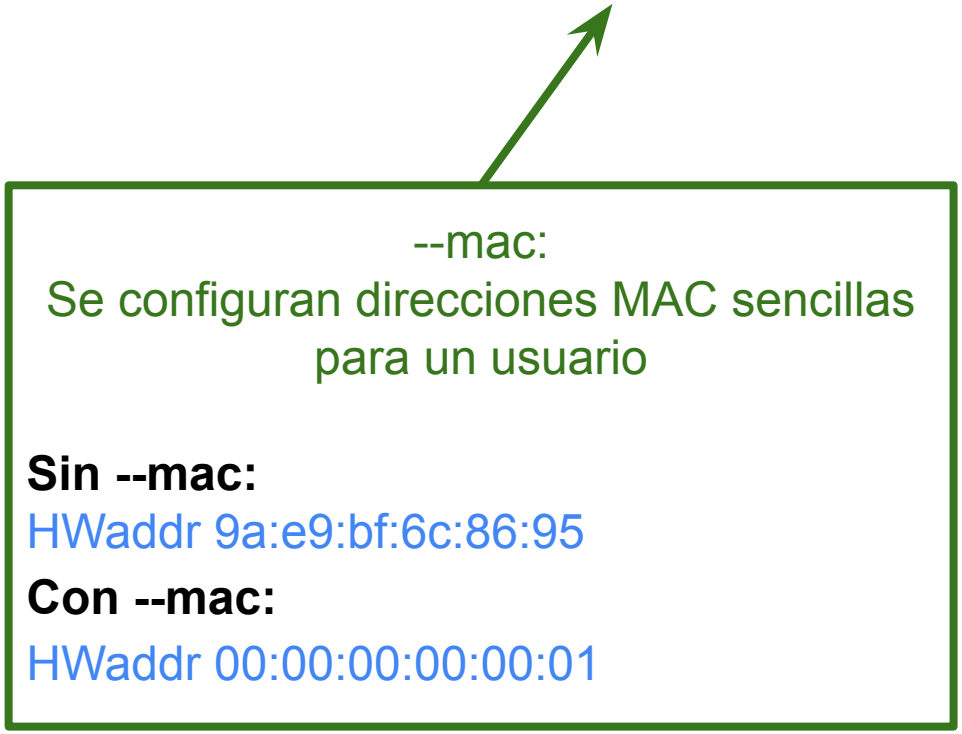
Mininet - Topologías custom

```
➜  ~ sudo mn --custom /tmp/topology/example.py --topo example --mac --switch
ovsk
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
sw1 sw2
*** Adding links:
(h1, sw1) (sw1, sw2) (sw2, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
sw1 sw2 ...
```



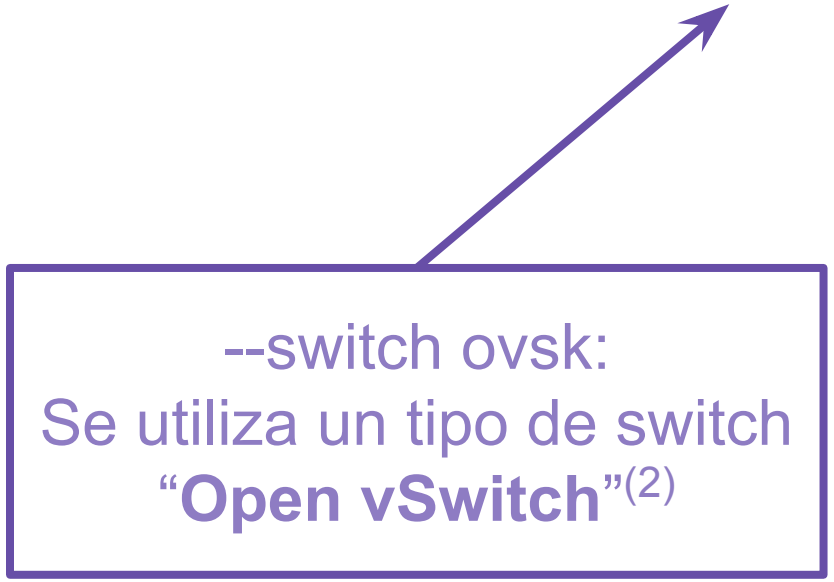
Mininet - Topologías custom

```
➜  ~ sudo mn --custom /tmp/topology/example.py --topo example --mac
--switch ovsk
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
sw1 sw2
*** Adding links:
(h1, sw1) (sw1, sw2) (sw2, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
sw1 sw2 ...
```



Mininet - Topologías custom

```
➜  ~ sudo mn --custom /tmp/topology/example.py --topo example --mac --switch ovsk
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
sw1 sw2
*** Adding links:
(h1, sw1) (sw1, sw2) (sw2, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
sw1 sw2 ...
```



(2) http://www.openvswitch.org/

Mininet

- ¿Es posible levantar un servidor?
- ¿Cómo hago que un host le haga un request a otro host/servidor?

Mininet - iperf

➜ **~ man iperf**

Es una herramienta para realizar mediciones de rendimiento de la red. Puede probar el rendimiento TCP o UDP. Para realizar una prueba iperf, el usuario debe establecer tanto un servidor (para descartar tráfico) como un cliente (para generar tráfico).

Mininet - iperf

Modo servidor:

➜ ~ sudo iperf -s -p 80

Server listening on TCP port 80
TCP window size: 85.3 KByte (default)

Modo cliente:

➜ ~ sudo iperf -c 10.0.0.1 -p 80

Client connecting to localhost, TCP port 80
TCP window size: 2.50 MByte (default)

---

[3] local 127.0.0.1 port 48512 connected with 127.0.0.1 port 80
[ID] Interval Transfer Bandwidth
[3] 0.0-10.0 sec 56.9 GBytes 48.9 Gbits/sec

Mininet - iperf

Desde la consola de Mininet

mininet> iperf h1 h2
***** Iperf: testing TCP bandwidth between h1 and h2**
***** Results: ['24.9 Gbits/sec', '25.0 Gbits/sec']**

Mininet

- ¿Y como hago para saber a que router se tiene que mandar un paquete?
- ¿Quién controla los routers?

Mininet - Controlador remoto

```
from mininet.topo import Topo

class MyTopo( Topo ):

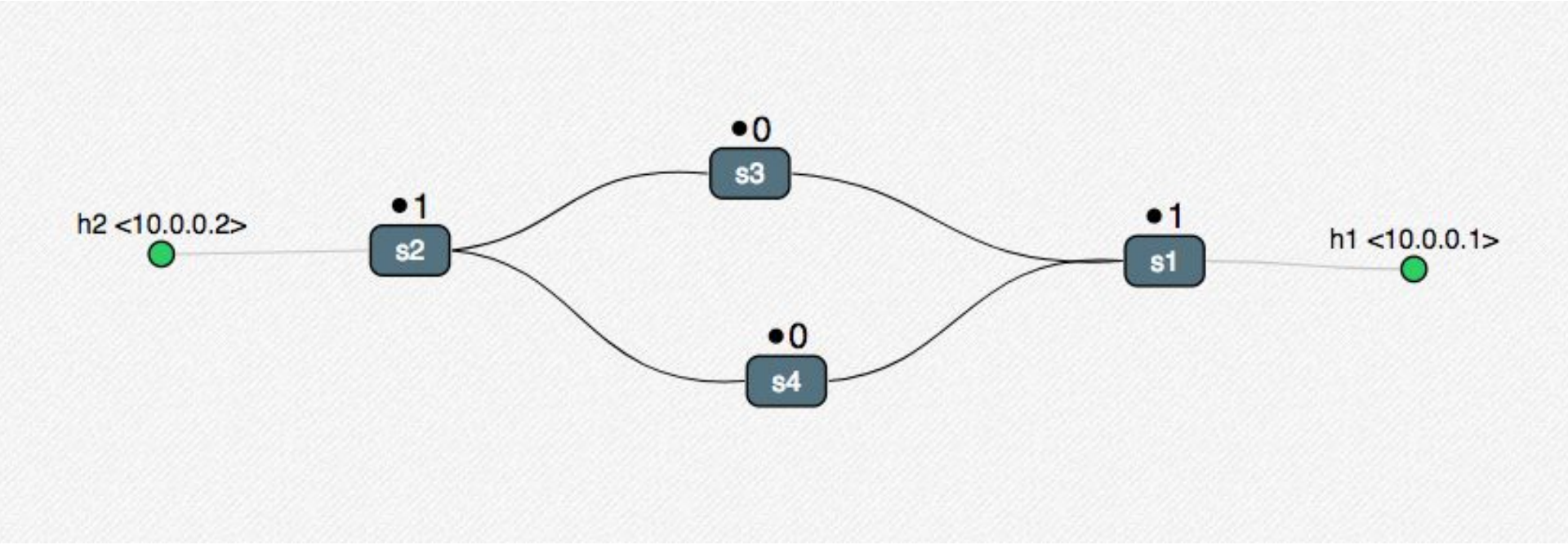
    def __init__( self ):
        # Initialize topology
        Topo.__init__( self )
        # Add hosts and switches
        leftHost = self.addHost( 'h1' )
        rightHost = self.addHost( 'h2' )
        leftSwitch = self.addSwitch( 's1' )
        rightSwitch = self.addSwitch( 's2' )
        mid1 = self.addSwitch('s3')
        mid2 = self.addSwitch('s4')

        # Add links
        self.addLink( leftHost, leftSwitch )
        self.addLink( rightHost, rightSwitch )
        self.addLink( leftSwitch, mid1 )
        self.addLink( leftSwitch, mid2 )
        self.addLink( rightSwitch, mid1 )
        self.addLink( rightSwitch, mid2 )

topos = { 'mytopo': ( lambda: MyTopo() ) }
```

custom/loop.py

Mininet - Controlador remoto



Mininet - Controlador remoto

```
➜  ~ sudo mn --custom ~/mininet/custom/loop.py --topo mytopo --arp --mac --switch ovsk --test pingall

*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (s1, s3) (s1, s4) (s2, s3) (s2, s4)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Waiting for switches to connect
s1 s2 s3 s4
*** Ping: testing ping reachability
h1 -> X
h2 -> X
*** Results: 100% dropped (0/2 received)
*** Stopping 1 controllers
c0
*** Stopping 6 links
.....
*** Stopping 4 switches
s1 s2 s3 s4
*** Stopping 2 hosts
h1 h2
*** Done
completed in 26.096 seconds
```

Mininet - POX

POX es una plataforma para desarrollar controladores SDN con fines académicos y de investigación.[3]

Volviendo al primer ejemplo

```
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s1-eth2 (OK OK)

mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=1907>
<Host h2: h2-eth0:10.0.0.2 pid=1909>
<OVSSwitch s1:
lo:127.0.0.1,s1-eth1:None,s1-eth2:None
pid=1914>
<Controller c0: 127.0.0.1:6653 pid=1900>
```

Mininet se conecta al controlador mediante la dirección y puerto indicados (por defecto 127.0.0.1:6653). Por lo que debemos correr el controlador escuchando en dicho socket.

(3) https://github.com/mininet/openflow-tutorial/wiki/Create-a-Learning-Switch#Controller_Choice_POX_Python

Mininet - Controlador remoto

```
➜  ~ pox/pox.py samples.spanning_tree
POX 0.1.0 (betta) / Copyright 2011-2013 James
McCauley, et al.
[core                  ] POX 0.1.0 (betta) is up.
[openflow.of_01        ] [1] connected
[openflow.of_01        ] [3] connected
[openflow.of_01        ] [4] connected
[openflow.of_01        ] [2] connected
[openflow.discovery ] link detected: 3 -> 1
[openflow.spanning_tree] 5 ports changed
```

```
sudo mn --custom ~/mininet/custom/loop.py
--topo mytopo --arp --mac --switch ovsk --test
pingall  --controller remote
...
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
...
```

Mininet - Controlador remoto

```
➜  ~ pox/pox.py samples.spanning_tree
POX 0.1.0 (betta) / Copyright 2011-2013 James
McCauley, et al.
[core                  ] POX 0.1.0 (betta) is up.
[openflow.of_01        ] [1] connected
[openflow.of_01        ] [3] connected
[openflow.of_01        ] [4] connected
[openflow.of_01        ] [2] connected
[openflow.discovery    ] link detected: 3 -> 1
[openflow.spanning_tree] 5 ports changed
```

openflow.discovery

Se corre un proceso mediante el cual cada switch aprende toda la topología de red mediante el protocolo LLDP - l2_learning switch

Mininet - Controlador remoto

```
➜  ~ pox/pox.py samples.spanning_tree
POX 0.1.0 (betta) / Copyright 2011-2013 James
McCauley, et al.
[core                  ] POX 0.1.0 (betta) is up.
[openflow.of_01        ] [1] connected
[openflow.of_01        ] [3] connected
[openflow.of_01        ] [4] connected
[openflow.of_01        ] [2] connected
[openflow.discovery    ] link detected: 3 -> 1
[openflow.spanning_tree] 5 ports changed
```

openflow.spanning_tree

Además del proceso de aprendizaje, se corre un proceso de spanning tree para evitar los bucles

Mininet - Controlador remoto

```
➜  ~ pox/pox.py samples.spanning_tree
POX 0.1.0 (betta) / Copyright 2011-2013 James
McCauley, et al.
[core                  ] POX 0.1.0 (betta) is up.
[openflow.of_01        ] [1] connected
[openflow.of_01        ] [3] connected
[openflow.of_01        ] [4] connected
[openflow.of_01        ] [2] connected
[openflow.discovery    ] link detected: 3 -> 1
[openflow.spanning_tree] 5 ports changed
```

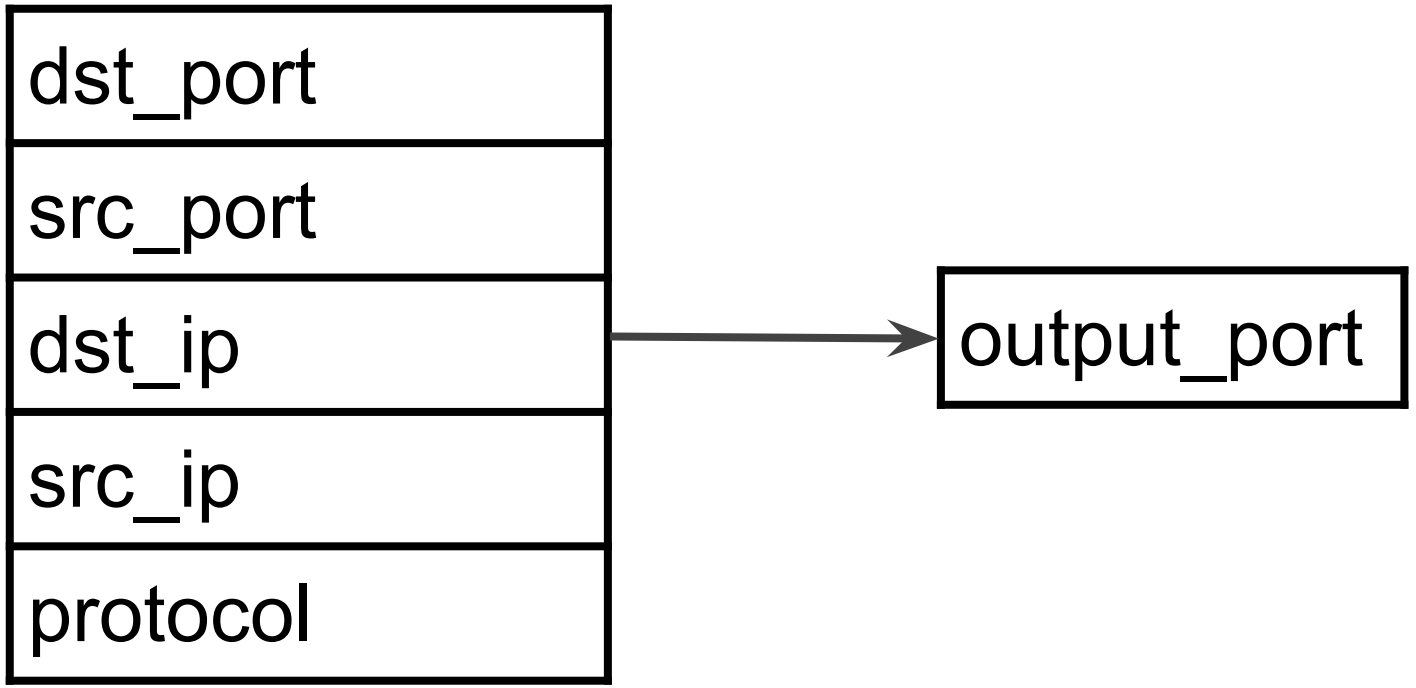
openflow.spanning_tree

Además del proceso de aprendizaje, se corre un proceso de spanning tree para evitar los bucles

IMPORTANTE

EN EL TP NO DEBEN USAR SPANNING TREE

¿Cómo definimos un flujo?



TP 2 - Entrega

- **5 personas.**
- Fecha Entrega: martes 25/06/2026 - 18:00 hs
- Formato Entrega:
  - Informe y Código por campus - Una por grupo
- Pruebas:
  - iperf
  - ping
  - Wireshark